

Experiment 8: Write a Program to perform Log Transformation of an image.

Aim

To write a MATLAB program to perform **log transformation** on a digital image for enhancing low-intensity pixels.

Objective

- To understand the concept of log transformation in image processing.
- To enhance the dark regions of an image.
- To apply logarithmic transformation to improve image visibility.
- To compare the original image with the log-transformed image.

Theory

Log transformation is a point processing image enhancement technique used to expand the values of dark pixels while compressing the values of bright pixels. It is particularly useful for images with a large dynamic range.

Log transformation:

- Enhances details in darker regions.
- Compresses higher intensity values.
- Improves image visibility without significantly affecting bright regions.

Algorithm

1. Start MATLAB.
2. Read the input image using `imread()`.
3. Convert the image to grayscale if it is a color image.
4. Convert the image to double precision using `im2double()`.
5. Apply the logarithmic transformation using the formula $s = c \log_{f_0}(1+r)$ $s = c \log(1 + r)$.
6. Display the original image and the transformed image.
7. Stop.

MATLAB Program

```
% Log Transformation of an Image
```

```
clc;  
clear;  
close all;
```

```
% Read the input image  
img = imread('cameraman.tif');
```

```
% Convert RGB image to grayscale if necessary  
if size(img,3) == 3
```

```

    img = rgb2gray(img);
end

% Convert image to double precision
img = im2double(img);

% Calculate scaling constant
c = 1;

% Apply Log Transformation
logImage = c * log(1 + img);

% Normalize the transformed image
logImage = mat2gray(logImage);

% Display images
figure;

subplot(1,2,1);
imshow(img);
title('Original Image');

subplot(1,2,2);
imshow(logImage);
title('Log Transformed Image');

```

Output

The program displays:

1. **Original Image** – The input grayscale image.
2. **Log Transformed Image** – The enhanced image in which dark regions become brighter and more visible while bright regions are compressed.

Viva Questions

1. What is log transformation?
2. Why is log transformation used in image processing?
3. Write the mathematical expression for log transformation.
4. What is the role of the scaling constant ccc?
5. Which regions of an image are enhanced by log transformation?
6. Which MATLAB function converts an image to double precision?
7. Why is mat2gray() used after log transformation?
8. What is the difference between log transformation and gamma transformation?
9. Name two applications of log transformation.
10. Is log transformation a point processing technique?

Experiment 9: Write a Program to implement Ideal low pass filter.

Aim

To write a MATLAB program to implement an **Ideal Low Pass Filter (ILPF)** in the frequency domain for smoothing a digital image.

Objective

- To understand the concept of frequency domain filtering.
- To implement an Ideal Low Pass Filter using MATLAB.
- To remove high-frequency components (noise and edges) from an image.
- To compare the original image with the filtered image.

Theory

An **Ideal Low Pass Filter (ILPF)** is a frequency domain filter that allows **low-frequency components** of an image to pass unchanged while completely blocking **high-frequency components**.

The ILPF is mainly used for:

- Image smoothing
- Noise reduction
- Blur generation
- Preprocessing in computer vision

Algorithm

1. Start MATLAB.
2. Read the input image.
3. Convert the image to grayscale if necessary.
4. Compute the Fourier Transform using `fft2()`.
5. Shift the zero-frequency component to the center using `fftshift()`.
6. Create an Ideal Low Pass Filter mask with the desired cutoff frequency.
7. Multiply the filter mask with the Fourier spectrum.
8. Apply the inverse Fourier Transform.
9. Display the original and filtered images.
10. Stop.

MATLAB Program

```
% Experiment 9
% Ideal Low Pass Filter

clc;
clear;
close all;

% Read image
img = imread('cameraman.tif');

% Convert RGB to grayscale if required
```

```

if size(img,3)==3
    img = rgb2gray(img);
end

% Convert image to double
img = im2double(img);

% Image size
[M,N] = size(img);

% Fourier Transform
F = fft2(img);
F = fftshift(F);

% Cutoff Frequency
D0 = 40;

% Create Ideal Low Pass Filter
H = zeros(M,N);

for u = 1:M
    for v = 1:N
        D = sqrt((u-M/2)^2 + (v-N/2)^2);
        if D <= D0
            H(u,v) = 1;
        end
    end
end

% Apply filter
G = H .* F;

% Inverse Fourier Transform
G = ifftshift(G);
output = real(ifft2(G));

% Display images
figure;

subplot(1,2,1);
imshow(img,[]);
title('Original Image');

subplot(1,2,2);
imshow(output,[]);
title('Ideal Low Pass Filter Output');

```

Output

The program displays:

1. **Original Image**
2. **Filtered Image using Ideal Low Pass Filter**

The filtered image appears smoother because high-frequency components such as sharp edges and noise are removed.

Viva Questions

1. What is an Ideal Low Pass Filter?
2. What is the purpose of an ILPF?
3. What is the cutoff frequency?
4. Why is the Fourier Transform used in frequency domain filtering?
5. Which MATLAB function computes the Fourier Transform?
6. Why is fftshift() used?

Experiment 10: Write a Program to implement Butterworth low pass filter.

Aim

To write a MATLAB program to implement a **Butterworth Low Pass Filter (BLPF)** in the frequency domain for smoothing a digital image while preserving more image details than an Ideal Low Pass Filter.

Objective

- To understand the concept of the Butterworth Low Pass Filter.
- To implement a Butterworth Low Pass Filter using MATLAB.
- To reduce high-frequency noise while minimizing ringing effects.
- To compare the original image with the filtered image.

Theory

A **Butterworth Low Pass Filter (BLPF)** is a frequency domain filter that allows low-frequency components to pass while gradually attenuating high-frequency components. Unlike the Ideal Low Pass Filter, the Butterworth filter has a smooth transition between the passband and stopband, thereby reducing ringing artifacts.

A higher filter order results in a sharper cutoff, while a lower order provides a smoother transition.

Algorithm

1. Start MATLAB.
2. Read the input image.
3. Convert the image to grayscale if necessary.
4. Compute the Fourier Transform using `fft2()`.
5. Shift the zero-frequency component to the center using `fftshift()`.
6. Create the Butterworth Low Pass Filter using the cutoff frequency and filter order.
7. Multiply the filter with the Fourier spectrum.
8. Apply the inverse Fourier Transform.
9. Display the original and filtered images.
10. Stop.

MATLAB Program

```
% Butterworth Low Pass Filter
```

```
clc;  
clear;  
close all;
```

```
% Read input image  
img = imread('cameraman.tif');
```

```
% Convert RGB image to grayscale if required  
if size(img,3)==3  
    img = rgb2gray(img);  
end
```

```

% Convert image to double
img = im2double(img);

% Image size
[M,N] = size(img);

% Fourier Transform
F = fft2(img);
F = fftshift(F);

% Cutoff frequency and filter order
D0 = 40;
n = 2;

% Create Butterworth Low Pass Filter
H = zeros(M,N);

for u = 1:M
    for v = 1:N
        D = sqrt((u-M/2)^2 + (v-N/2)^2);
        H(u,v) = 1 / (1 + (D/D0)^(2*n));
    end
end

% Apply filter
G = H .* F;

% Inverse Fourier Transform
G = ifftshift(G);
output = real(ifft2(G));

% Display images
figure;

subplot(1,2,1);
imshow(img,[]);
title('Original Image');

subplot(1,2,2);
imshow(output,[]);
title('Butterworth Low Pass Filter Output');

```

Output

The program displays:

1. **Original Image**
2. **Butterworth Low Pass Filtered Image**

The filtered image appears smoother than the original, with reduced high-frequency noise while preserving more image details compared to the Ideal Low Pass Filter.

Viva Questions

1. What is a Butterworth Low Pass Filter?
2. How does a Butterworth filter differ from an Ideal Low Pass Filter?
3. What is the role of the cutoff frequency ω_c ?
4. What is the significance of the filter order n ?
5. Why is the Fourier Transform used in frequency domain filtering?
6. What is the purpose of `fftshift()`?

Experiment No. 11: Implement Gaussian Low Pass Filter (GLPF)

Aim

To write a MATLAB program to implement a **Gaussian Low Pass Filter (GLPF)** in the frequency domain for smoothing a digital image and reducing high-frequency noise.

Objective

- To understand the concept of the Gaussian Low Pass Filter.
- To implement a Gaussian Low Pass Filter using MATLAB.
- To remove high-frequency noise while preserving image quality.
- To compare the original image with the Gaussian-filtered image.

Theory

A **Gaussian Low Pass Filter (GLPF)** is a frequency domain filter that attenuates high-frequency components according to a Gaussian function. Unlike the Ideal and Butterworth filters, the Gaussian filter provides a very smooth transition between the passband and stopband, thereby eliminating ringing (Gibbs) artifacts.

As the distance from the center increases, the filter response decreases smoothly according to the Gaussian curve.

Algorithm

1. Start MATLAB.
2. Read the input image using `imread()`.
3. Convert the image to grayscale if it is a color image.
4. Convert the image to double precision.
5. Compute the Fourier Transform using `fft2()`.
6. Shift the frequency spectrum to the center using `fftshift()`.
7. Generate the Gaussian Low Pass Filter mask.
8. Multiply the filter mask with the Fourier spectrum.
9. Apply the inverse Fourier Transform.
10. Display the original and filtered images.
11. Stop.

MATLAB Program

```
% Gaussian Low Pass Filter
```

```
clc;  
clear;  
close all;
```

```
% Read input image  
img = imread('cameraman.tif');
```

```
% Convert RGB image to grayscale if required  
if size(img,3)==3  
    img = rgb2gray(img);
```

```

end

% Convert image to double precision
img = im2double(img);

% Image size
[M,N] = size(img);

% Fourier Transform
F = fft2(img);
F = fftshift(F);

% Cutoff frequency
D0 = 40;

% Create Gaussian Low Pass Filter
H = zeros(M,N);

for u = 1:M
    for v = 1:N
        D = sqrt((u-M/2)^2 + (v-N/2)^2);
        H(u,v) = exp(-(D^2)/(2*(D0^2)));
    end
end

% Apply filter
G = H .* F;

% Inverse Fourier Transform
G = ifftshift(G);
output = real(ifft2(G));

% Display images
figure;

subplot(1,2,1);
imshow(img,[]);
title('Original Image');

subplot(1,2,2);
imshow(output,[]);
title('Gaussian Low Pass Filter Output');

```

Output

The program displays:

1. **Original Image**
2. **Gaussian Low Pass Filtered Image**

The filtered image appears smoother with reduced noise while preserving image structures better than the Ideal Low Pass Filter. It also avoids the ringing artifacts commonly observed with the Ideal filter.

Viva Questions

1. What is a Gaussian Low Pass Filter?
2. Write the mathematical expression of the Gaussian Low Pass Filter.
3. What is the role of the cutoff frequency (D_0/D_0)?
4. Why does the Gaussian filter not produce ringing artifacts?
5. Which MATLAB function is used to compute the Fourier Transform?
6. Why is `fftshift()` used?

Experiment 12: Write a Program for generating noise PDFs for uniform, Rayleigh and exponential noise.

Aim

To write a MATLAB program to generate and plot the Probability Density Functions (PDFs) of Uniform, Rayleigh, and Exponential noise distributions.

Objective

- To understand different types of noise used in digital image processing.
- To generate Uniform, Rayleigh, and Exponential noise using MATLAB.
- To plot and analyze the Probability Density Functions (PDFs) of these noise models.
- To compare the characteristics of different noise distributions.

Theory

In digital image processing, noise is an unwanted random variation in pixel intensity that degrades image quality. Different imaging systems introduce different types of noise. The most commonly used noise models are Uniform, Rayleigh, and Exponential noise.

1. Uniform Noise

Uniform noise has a constant probability over a specified interval.

2. Rayleigh Noise

Rayleigh noise is commonly used in radar and ultrasound imaging.

3. Exponential Noise

Exponential noise is often used to model random events occurring independently over time.

Algorithm

1. Start MATLAB.
2. Define the required range of values.
3. Generate the Uniform PDF.
4. Generate the Rayleigh PDF.
5. Generate the Exponential PDF.
6. Plot all three PDFs separately.
7. Label the axes and titles.
8. Observe the differences in the distributions.
9. Stop.

MATLAB Program

```
% Experiment 12
% Generate Uniform, Rayleigh and Exponential Noise PDFs

clc;
```

```

clear;
close all;

%% Uniform PDF
x = -2:0.01:2;
a = -1;
b = 1;

uniform_pdf = zeros(size(x));
uniform_pdf(x>=a & x<=b) = 1/(b-a);

figure;
plot(x,uniform_pdf,'LineWidth',2);
grid on;
title('Uniform Noise PDF');
xlabel('Noise Value');
ylabel('Probability Density');

%% Rayleigh PDF
x = 0:0.01:5;
sigma = 1;

rayleigh_pdf = (x/(sigma^2)).*exp(-(x.^2)/(2*sigma^2));

figure;
plot(x,rayleigh_pdf,'LineWidth',2);
grid on;
title('Rayleigh Noise PDF');
xlabel('Noise Value');
ylabel('Probability Density');

%% Exponential PDF
x = 0:0.01:5;
lambda = 1;

exp_pdf = lambda*exp(-lambda*x);

figure;
plot(x,exp_pdf,'LineWidth',2);
grid on;
title('Exponential Noise PDF');
xlabel('Noise Value');
ylabel('Probability Density');

```

Output

The program generates three separate graphs:

1. Uniform Noise PDF
 - A rectangular distribution with constant probability between the specified limits.
2. Rayleigh Noise PDF
 - A skewed distribution that rises to a peak and then gradually decreases.
3. Exponential Noise PDF

- A distribution with the highest probability at zero that decreases exponentially as the noise value increases.

Viva Questions

1. What is noise in digital image processing?
2. What is a Probability Density Function (PDF)?
3. Which distribution has a constant probability?
4. Where is Rayleigh noise commonly observed?
5. What is the shape of the Exponential PDF?
6. What is the role of the parameter σ (**sigma**) in the Rayleigh distribution?

Experiment 13: Write a Program to implement various edge detection operators.

Aim

To write a MATLAB program to implement various **edge detection operators** such as **Sobel, Prewitt, Roberts, Canny, and Laplacian of Gaussian (LoG)** for detecting edges in a digital image.

Objective

- To understand the concept of edge detection in digital image processing.
- To implement different edge detection operators using MATLAB.
- To compare the performance of various edge detection techniques.
- To identify object boundaries and significant image features.

Theory

Edge detection is one of the most important image processing operations used to identify points in an image where the intensity changes abruptly. These discontinuities generally correspond to object boundaries, corners, and other important features.

MATLAB provides the **edge()** function to detect edges using different algorithms.

Algorithm

1. Start MATLAB.
2. Read the input image using `imread()`.
3. Convert the image to grayscale if it is a color image.
4. Apply the Sobel operator.
5. Apply the Prewitt operator.
6. Apply the Roberts operator.
7. Apply the Canny operator.
8. Apply the LoG operator.
9. Display the original image along with all detected edge images.
10. Stop.

MATLAB Program

% Edge Detection using Various Operators

```
clc;  
clear;  
close all;
```

```
% Read input image  
img = imread('cameraman.tif');
```

```
% Convert RGB image to grayscale if required  
if size(img,3) == 3  
    img = rgb2gray(img);  
end
```

```
% Edge Detection
```

```

sobelEdge = edge(img,'sobel');
prewittEdge = edge(img,'prewitt');
robertsEdge = edge(img,'roberts');
cannyEdge = edge(img,'canny');
logEdge = edge(img,'log');

% Display Results
figure;

subplot(2,3,1);
imshow(img);
title('Original Image');

subplot(2,3,2);
imshow(sobelEdge);
title('Sobel');

subplot(2,3,3);
imshow(prewittEdge);
title('Prewitt');

subplot(2,3,4);
imshow(robertsEdge);
title('Roberts');

subplot(2,3,5);
imshow(cannyEdge);
title('Canny');

subplot(2,3,6);
imshow(logEdge);
title('LoG');

```

Output

The MATLAB program displays **six images** in a single figure:

1. Original Image
2. Sobel Edge Detection
3. Prewitt Edge Detection
4. Roberts Edge Detection
5. Canny Edge Detection
6. Laplacian of Gaussian (LoG) Edge Detection

Each operator highlights the edges differently based on its detection technique.

Viva Questions

1. What is edge detection?
2. Why is edge detection important in image processing?
3. Which MATLAB function is used for edge detection?
4. Name five edge detection operators.
5. Which operator provides the best noise immunity?

Experiment 14: Open Ended program

Aim

To develop a MATLAB program for **image sharpening using the Unsharp Masking technique** to enhance image details and edges.

Objective

- To understand the concept of image sharpening.
- To implement the Unsharp Masking technique in MATLAB.
- To enhance edges and fine details in a digital image.
- To compare the original image with the sharpened image.

Theory

Image sharpening is an image enhancement technique used to improve the visibility of edges and fine details. One of the most commonly used methods is **Unsharp Masking**, which enhances an image by emphasizing its high-frequency components.

The process involves:

1. Blurring the original image using a Gaussian filter.
2. Subtracting the blurred image from the original image to obtain the high-frequency details (mask).
3. Adding the mask back to the original image to produce a sharpened image.

This technique improves image clarity while preserving important visual features.

Algorithm

1. Start MATLAB.
2. Read the input image using `imread()`.
3. Convert the image to grayscale if necessary.
4. Apply Gaussian smoothing using `imgaussfilt()`.
5. Compute the unsharp mask by subtracting the blurred image from the original image.
6. Add the mask to the original image.
7. Display the original, blurred, and sharpened images.
8. Stop.

MATLAB Program

% Open-Ended Program: Image Sharpening Using Unsharp Masking

```
clc;  
clear;  
close all;
```

```
% Read the input image  
img = imread('cameraman.tif');
```

```
% Convert RGB image to grayscale if required
```

```

if size(img,3) == 3
    img = rgb2gray(img);
end

% Convert image to double
img = im2double(img);

% Apply Gaussian Blur
blurred = imgaussfilt(img, 2);

% Generate Unsharp Mask
mask = img - blurred;

% Sharpen the image
k = 1.5; % Sharpening factor
sharpened = img + k * mask;

% Limit pixel values to [0,1]
sharpened = min(max(sharpened,0),1);

% Display Results
figure;

subplot(1,3,1);
imshow(img);
title('Original Image');

subplot(1,3,2);
imshow(blurred);
title('Blurred Image');

subplot(1,3,3);
imshow(sharpened);
title('Sharpened Image');

```

Output

The MATLAB program displays three images:

1. **Original Image** – The input grayscale image.
2. **Blurred Image** – The image after Gaussian smoothing.
3. **Sharpened Image** – The enhanced image with improved edge details.

Viva Questions

1. What is image sharpening?
2. What is Unsharp Masking?
3. Why is Gaussian filtering used before sharpening?
4. What is the purpose of the sharpening factor (**k**)?
5. Which MATLAB function is used for Gaussian filtering?
6. What is the difference between image smoothing and image sharpening?
7. What are high-frequency components in an image?